

CE 206: Engineering Computation Sessional

Finding Roots of Equations

Department of Civil Engineering

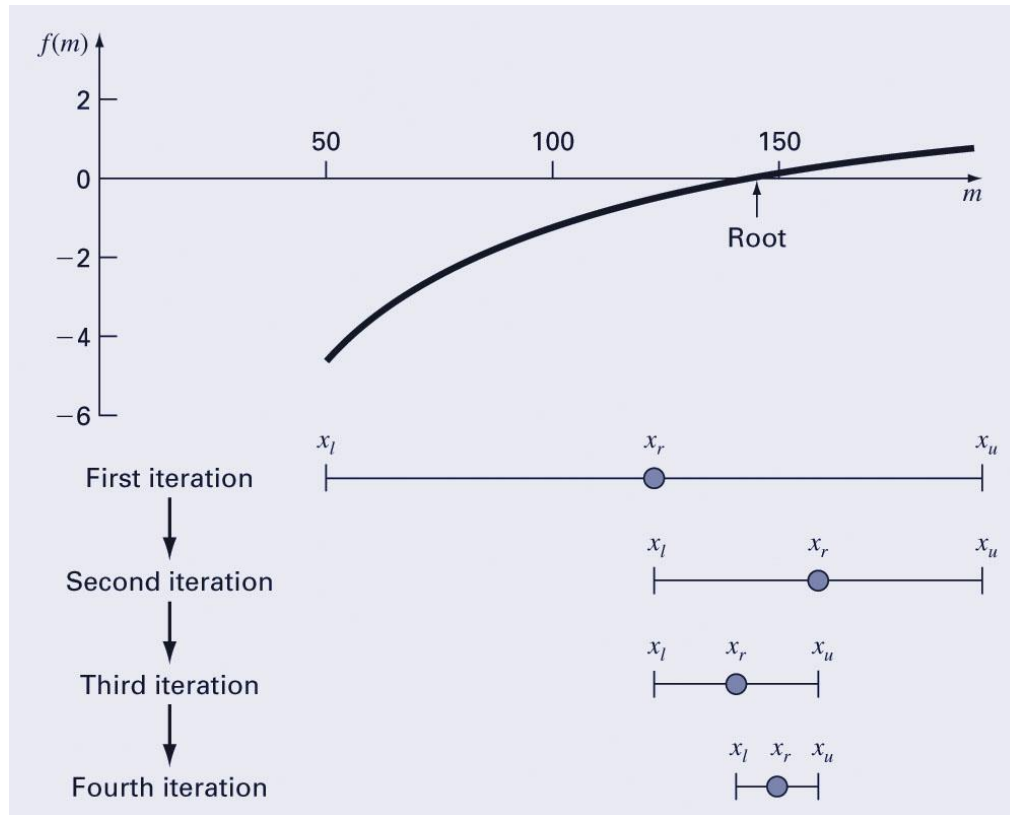
Solution Methods Overview

- Bisection/Half-interval Search
 - Method of false position/Regula Falsi
 - Secant Method
 - Newton Raphson
 - Iteration Method
 - Many more....
- Bracketing Methods
- Open Methods

Algebraic equation: $ax^2 + bx + c = 0$

Transcendental equation: $1 + \cos x - 5x = 0$

Bracketing Methods: Bisection



$$\mathcal{E}_a = \left| \frac{x_{new} - x_{old}}{x_{new}} \right| 100\%$$

Location of the root lies within the interval where the sign change occurs. The absolute error is reduced by a factor of 2 for each iteration.

MATLAB code: bisection method

```
function [root,fx,ea,iter]=bisection(func,xl,xu,es,maxit)
```

```
% [root,fx,ea,iter]=bisection('func',xl,xu,es,maxit):  
% uses bisection method to find the root of func  
% input:  
% func = name of function  
% xl, xu = lower and upper guesses  
% es = desired relative error (default = 0.0001)  
% maxit = maximum allowable iterations (default = 50)  
% output:  
% root = real root  
% fx = function value at root  
% ea = approximate relative error (%)  
% iter = number of iterations
```

*Comment
lines*

```
if nargin<3,error('at least 3 input arguments required'),end  
test = feval('func',xl)*feval('func',xu);  
if test>0,error('roots cannot be found'),end  
if nargin<4|isempty(es), es=0.0001;end  
if nargin<5|isempty(maxit), maxit=50;end
```

*Checking whether the
roots are within these
bounds or not*

MATLAB code: bisection method (ctd)

```
iter = 0; xr = xl; ea = 100;  
while (1)
```

The variables that change their values within the code needs to be initialized

```
    xrold = xr;
```

```
    xr = (xl + xu)/2;
```

```
    iter = iter + 1;
```

```
    if xr ~= 0, ea = abs((xr - xrold)/xr) * 100; end
```

Calculating approx. error

$$\mathcal{E}_a = \left| \frac{x_{new} - x_{old}}{x_{new}} \right| 100\%$$

```
    test = feval('func',xl)*feval('func',xr);
```

```
    if test < 0
```

```
        xu = xr;
```

```
    elseif test > 0
```

```
        xl = xr;
```

```
    else
```

```
        ea = 0;
```

```
    end
```

```
    if ea <= es | iter >= maxit, break, end
```

Stopping criteria

```
end
```

```
root = xr; fx = feval('func',xr);
```

func.m file definition

```
function [value] = func(x)
```

```
value = x^3-10*x^2+5; ← Evaluating the function  $x^3 - 10x^2 + 5$ 
```

```
[root,fx,ea,iter]=bisection('func',0.6,0.8)
```

```
root =  
    0.7346
```

← Root of the function after 19 iterations

```
fx =  
    0.4430
```

```
ea =  
    5.1929e-005 ←  $ea < 0.0001$  was the stopping criterion
```

```
iter =  
    19
```

bisection.m function only works with a function with name “func”

Exercise

Modify the `bisect.m` function file so that it can take function with any name or any form.

```
function [root,fx,ea,iter]= bisect_modified(func,xl,xu,es,maxit)

% [root,fx,ea,iter]=bisect('func',xl,xu,es,maxit):
% uses bisection method to find the root of func
% input:
% func = name of function
% xl, xu = lower and upper guesses
% es = desired relative error (default = 0.0001)
% maxit = maximum allowable iterations (default = 50)
% output:
% root = real root
% fx = function value at root
% ea = approximate relative error (%)
% iter = number of iterations

if nargin<3,error('at least 3 input arguments required'),end
test = func(xl)*func(xu);
if test>0,error('roots cannot be found'),end
if nargin<4|isempty(es), es=0.0001;end
if nargin<5|isempty(maxit), maxit=50;end
```

Exercise: bisection method (ctd)

```
iter = 0; xr = xl; ea = 100;
while (1)
    xrold = xr;
    xr = (xl + xu)/2;
    iter = iter + 1;
    if xr ~= 0,ea = abs((xr - xrold)/xr) * 100;end
    test = func(xl)*func(xr);
    if test < 0
        xu = xr;
    elseif test > 0
        xl = xr;
    else
        ea = 0;
    end
    if ea <= es | iter >= maxit,break,end
end
root = xr; fx = func(xr);
```


Passing a function as function argument

```
function [value] = my_new_func(x)
```

```
value = x^3-10*x^2+5; ← Evaluating the function  $x^3 - 10x^2 + 5$ 
```

Syntax 1:

```
[root,fx,ea,iter]=bisect_modified(@my_new_func,0.6,0.8)
```

Syntax 2:

```
[root,fx,ea,iter]=bisect_modified(@(x) x^3-10*x^2+5,0.6,0.8)
```

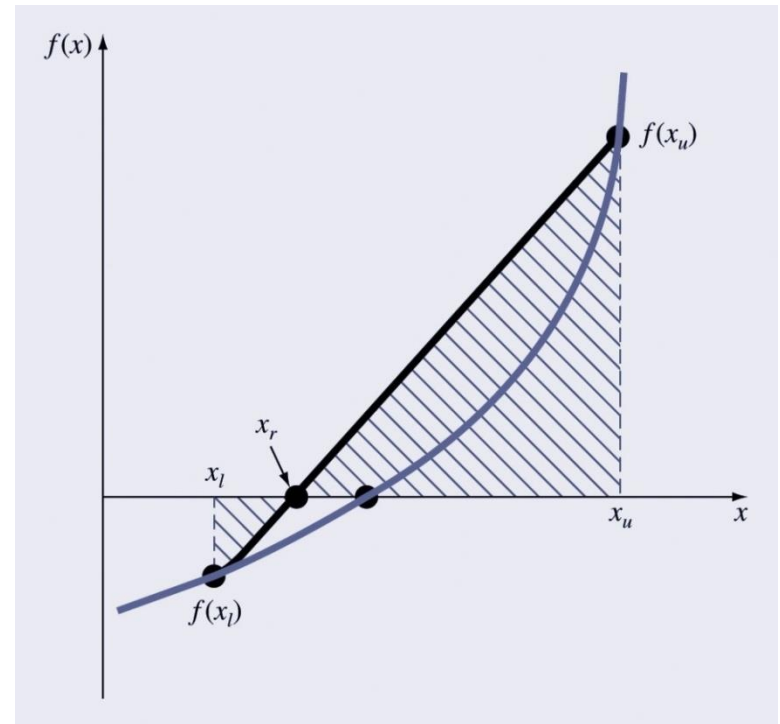
Bracketing Methods: Regula Falsi

the location of the intercept
of the straight line (x_r):

$$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$$

Try yourself:

Modify the **bisect.m**
function file to incorporate the
regula falsi method



bisection **xr = (xl + xu) / 2;**

Regula Falsi **xr = xu - feval('func', xu) * (xl - xu) / (feval('func', xl) - feval('func', xu));**

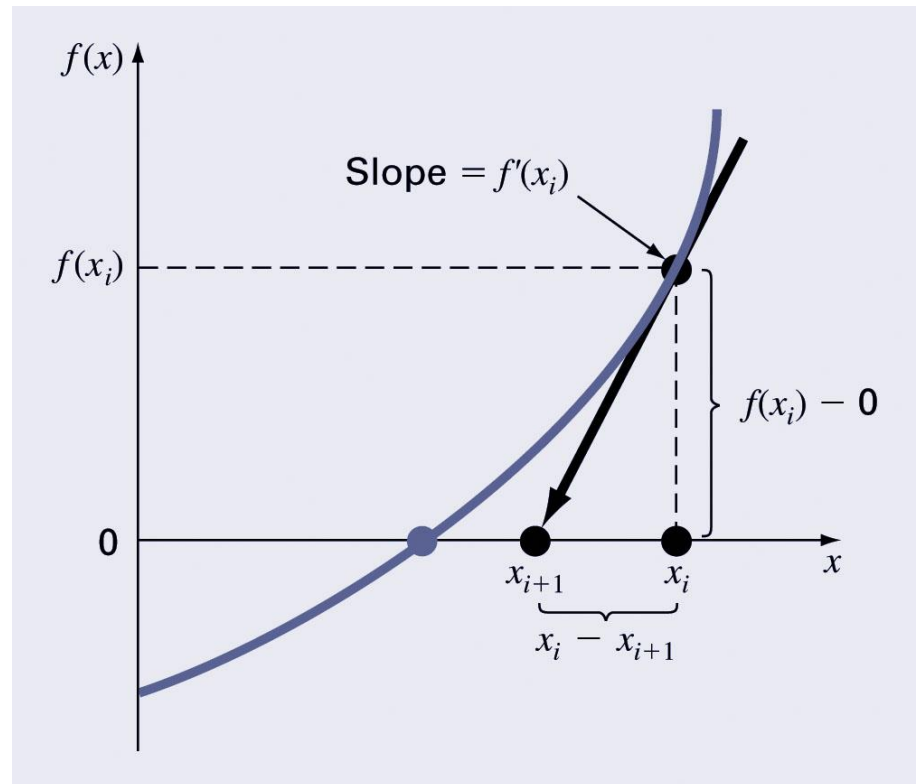
Newton-Raphson Method

Based on forming the tangent line to the $f(x)$ curve at some guess x , then following the tangent line to where it crosses the x -axis.

$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}}$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

$$\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| 100\%$$



MATLAB code: `newtraph.m`

```
function [root,ea,iter]=newtraph(func,dfunc,xr,es,maxit)

if nargin<3,
    error('at least 3 input arguments required')
end
if nargin<4|isempty(es),es=0.0001;end
if nargin<5|isempty(maxit),maxit=50;end
iter = 0;
while (1)
    xrold = xr;
    xr = xr - feval('func',xr)/feval('dfunc',xr);
    iter = iter + 1;
    if xr ~= 0, ea = abs((xr - xrold)/xr) * 100; end
    if ea <= es | iter >= maxit, break, end
end
root = xr;
```

Derivative of the function (needs to be defined)

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

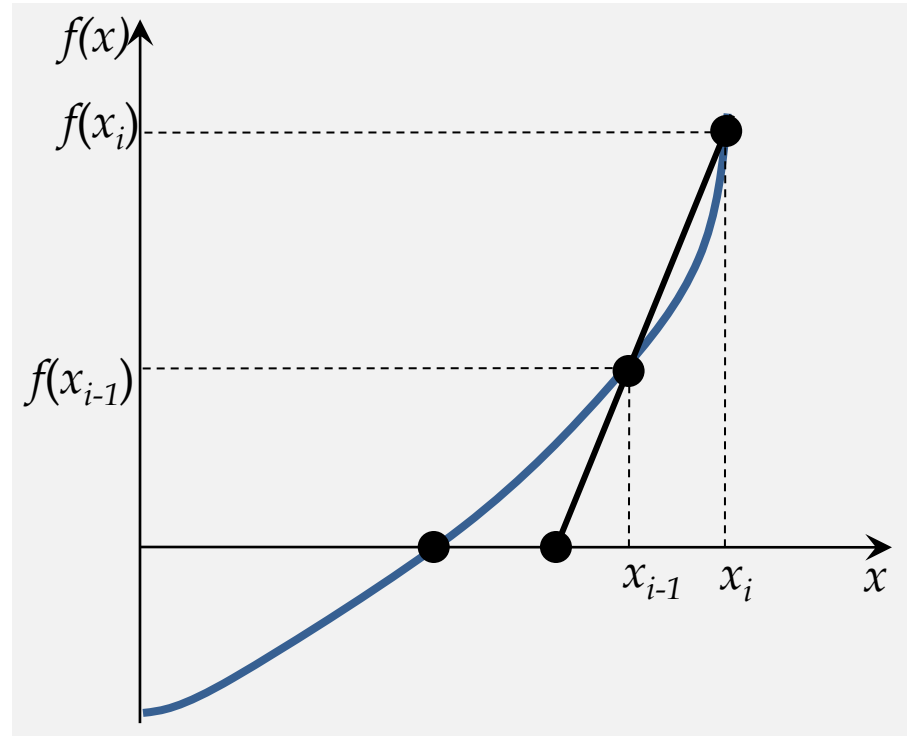
Secant Method

the derivative approximated by a backward finite divided difference:

$$f'(x_i) \cong \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i}$$

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

Note: Requires two initial guesses for x



Exercise:

Can you modify **newtraph.m** function file and make it applicable for the secant method?

Secant Method: Exercise

Apply the developed code for the secant method to find the first positive root of $f(x) = \sin x + \cos(1+x^2) - 1$ where x is in radians.

Use four iterations with initial guesses of

- (i) 1.0 and 3.0
- (ii) 1.5 and 2.5, and
- (iii) 1.5 and 2.25

MATLAB's **fzero** function

MATLAB's **fzero** provides the best qualities of both bracketing methods and open methods.

Using an initial guess:

```
x = fzero(function, x0)  
[x, fx] = fzero(function, x0)
```

*Location of
the root*

*Evaluated at
the root*

*Function handle to the
function being evaluated*

Initial guess

Using an initial bracket:

```
x = fzero(function, [x0 x1])  
[x, fx] = fzero(function, [x0 x1])
```

Initial guesses that bracket the root

Using create or edit **fzero** options using **optimset**

```
options = optimset('param1',value1,'param2',value2,...)
```

fzero example

```
options = optimset('display', 'iter');
```

- Sets options to display each iteration of root finding process.
- Other **optimset** properties: **MaxIter**, **TolX**

```
[x, fx] = fzero(@(x) x^10-1, 0.5, options)
```

- Uses fzero to find roots of $f(x) = x^{10} - 1$ starting with an initial guess of $x = 0.5$.

MATLAB reports $x = 1$, $fx = 0$ after 35 function counts

MATLAB's **roots** function

Roots of polynomials:

- For an nth order equation, there are n real or complex roots.

$$f_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

x = roots(c)

x is a column vector containing the roots

c is a row vector containing the polynomial coefficients

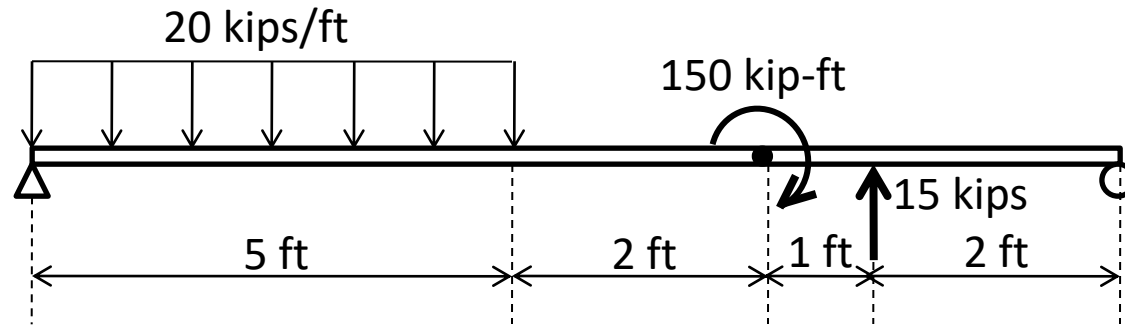
Example:

Find the roots of

$$f(x) = x^5 - 3.5x^4 + 2.75x^3 + 2.125x^2 - 3.875x + 1.25$$

x = roots([1 -3.5 2.75 2.125 -3.875 1.25])

Exercise 1



Using singularity functions, the shear, bending moment and displacement along the simply supported beam can be expressed as follows:

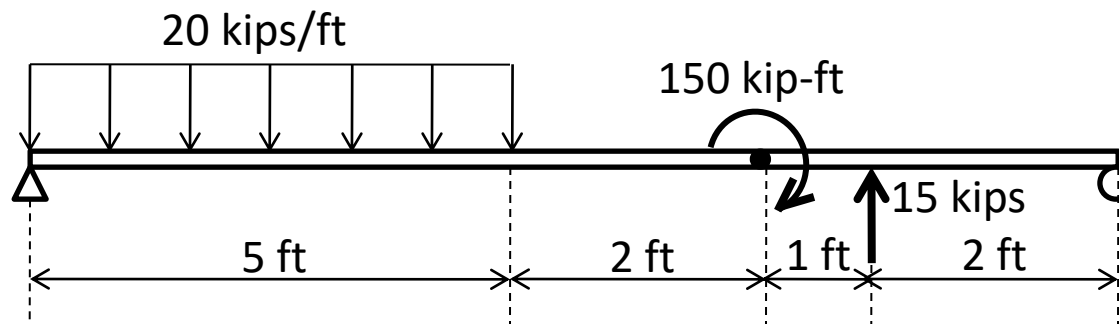
$$V(x) = 20\langle x-0 \rangle^1 - 20\langle x-5 \rangle^1 - 15\langle x-8 \rangle^0 - 57$$

$$M(x) = -10\langle x-0 \rangle^2 + 10\langle x-5 \rangle^2 + 15\langle x-8 \rangle^1 + 150\langle x-7 \rangle^0 + 57x$$

$$u(x) = -\frac{5}{6}\langle x-0 \rangle^4 + \frac{5}{6}\langle x-5 \rangle^4 + \frac{15}{6}\langle x-8 \rangle^2 + 75\langle x-7 \rangle^2 + \frac{57}{6}x^3 - 238.25x$$

Using any root location technique, find the points where shear and bending moment equal to zero and where the displacement is maximum. Also draw the shear force, bending moment and displacement diagrams

Exercise 1: solution hints



$$V(x) = 20\langle x-0 \rangle^1 - 20\langle x-5 \rangle^1 - 15\langle x-8 \rangle^0 - 57$$

```
function f = V(x)
f=20*(sing(x,0,1)-sing(x,5,1))-15*sing(x,8,0)-57;
```

In addition, the singularity function can be set up as

```
function s = sing(x, a, n)
if x > a
    s = (x - a) ^ n;
else
    s = 0;
end
```

$$\langle x-a \rangle^n = \begin{cases} (x-a)^n & \text{when } x > a \\ 0 & \text{when } x \leq a \end{cases}$$

**Solution: $V(x) = 0$ at $x = 2.85$ ft, $M(x) = 0$ at $x = 5.814$ ft,
 $U(x) = 0$ at $x = 3.9357$ ft**

Exercise 2

The saturation concentration of dissolved oxygen in freshwater can be calculated with the equation:

$$\ln O_{sat} = -139.34411 + \frac{1.575701 \times 10^5}{T_a} - \frac{6.642308 \times 10^7}{T_a^2} + \frac{1.2438 \times 10^{10}}{T_a^3} - \frac{8.621949 \times 10^{11}}{T_a^4}$$

O_{sat} = Saturation concentration of DO (mg/L)

$T_a = T + 273.15$ where T = temperature (°C)

Fill up the following table (use bisection method): express your results for an absolute error of 0.05 °C

O_{sat} (mg/L)	Temperature (°C)
8	??
10	??
12	??

Exercise 2: solution hints

First determine how many iterations you need for the specified absolute error using the formula (see your CE 205 notes):

$$n = \log_2 \left(\frac{\Delta x^0}{E_{a,d}} \right) = \log_2 \left(\frac{40}{0.05} \right) = 9.6439$$

Bracket range (points to Δx^0)

Absolute error (points to $E_{a,d}$)

10 iterations should be enough (points to 9.6439)

Use **bisect.m** to compute the roots for maximum 10 iterations and also find the real roots using **fzero** function. Compare results.

O_{sat}	Approximation	Exact	Error
8	26.75781	26.78017	0.0224
10	15.35156	15.38821	0.0366
12	7.46094	7.46519	0.0043

Less than 0.05 (points to the Error column)

Concluding remarks

Need to be judicious in choosing your initial guess (or initial brackets) of the root.

A physical insight into the problem may help

Apply common logic

Some examples:

- Water level in a spherical tank cannot exceed the diameter
- Temperature in natural water bodies usually within a specific range, (e.g. cannot be negative)
- deflection of a beam is maximum where the du/dx is zero
- Water level in a tank cannot exceed tank depth (or cannot be negative)
- etc.

Exercise

Water is flowing in a trapezoidal channel at a rate of $Q = 20 \text{ m}^3/\text{s}$. The critical depth y for such a channel must satisfy the equation:

$$0 = 1 - \frac{Q^2}{g A_c^3} B$$

Where $g = 9.81 \text{ m}^2/\text{s}$,

A_c = channel cross-sectional area (m^2) = $3y + y^2/2$ and

B = width of the channel at the surface (m) = $3 + y$.

Solve for the critical depth (y) using both bisection and method of false position using the initial guesses 0.5 and 2.5 and iterate until the approximate error falls below 1% or the number of iterations exceed 10. Compare your results from both methods

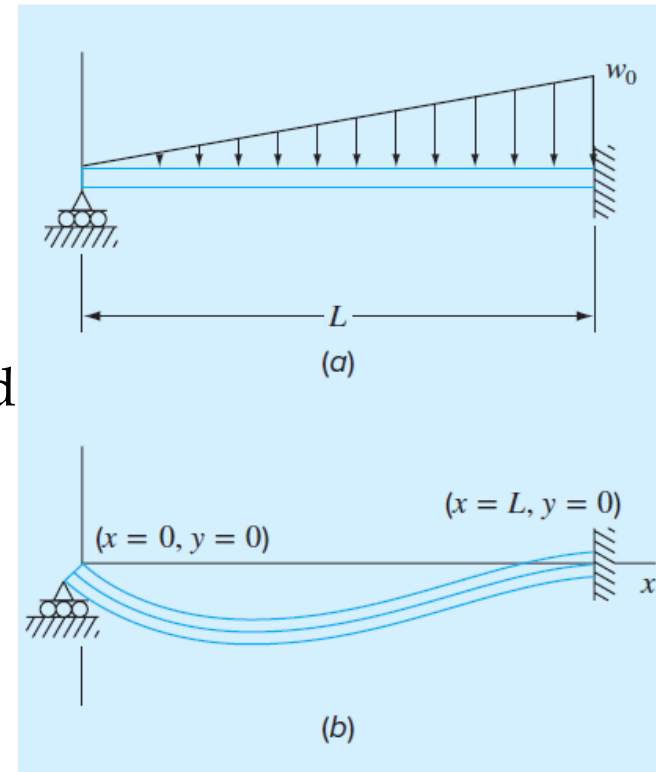
Assignment

A uniform beam is subjected to a linearly increasing distributed load. The equation for the resulting elastic curve is:

$$y = \frac{w_0}{120EIL} (-x^5 + 2L^2 x^3 - L^4 x)$$

Use Bisection, Regula Falsi, Newton-Raphson, and Secant methods to determine the point of maximum deflection (i.e., the value of x where $\frac{dy}{dx} = 0$). And determine the maximum deflection. Compare x , maximum deflection, and errors of these methods.

$L = 6 \cdot r$ cm, $E = 50,000 + 10,000 \cdot p$ kN/cm²,
 $I = 30,000$ cm⁴, and $w_0 = 2.5$ kN/cm



r = last 2 digits of your student no.
 p = 0 for odd and 1 for even student nos.